



P01-08 · T18 CI Lanes, Local Gates & Budget Instrumentation



The rule from the reliability pack, made concrete here: **budgets are assertions, not aspirations**. A performance number that lives in a wiki page is a wish. A performance number that fails a test is a contract. Every budget on the phase page becomes a failing test on this page.

1. `perf/budgets.toml` — the single source of truth

```
# Every budget in PHASE-01, machine-readable. Read by aura-pe  
rf at test time and  
# by CI to publish the trend. Nothing here is decoration: eac  
h entry has a test  
# that fails when the ceiling is exceeded.  
#  
# target = what we design for; a regression past this warns  
# ceiling = what we ship; a regression past this FAILS the bu  
ild  
#  
# Reference machine T-Typ: 8 physical cores, 32 GB RAM, NVMe  
SSD, 8 GB VRAM.  
# CI runners are slower, so CI multiplies every ceiling by `c  
i_slack`.
```

```
[meta]  
tier = "T-Typ"  
ci_slack = 1.6           # GitHub runners are ~1.6x slower o  
n IO-bound work
```

```
regression_warn_pct = 7    # a 7 % slide against the stored ba  
seline warns  
regression_fail_pct = 15   # 15 % fails, even if still inside  
the ceiling
```

```
[[budget]]  
id = "import_4000_files"  
what = "Import 4,000 files / 22 GB synthetic corpus, referenc  
e mode"  
unit = "seconds"  
target = 300  
ceiling = 540  
owner = "PERF"  
test = "aura-perf::budget_import_4000"
```

```
[[budget]]  
id = "hash_throughput"  
what = "Sustained BLAKE3 throughput, 8 threads, NVMe"  
unit = "MB/s"  
target = 700  
floor = 400                # a floor, not a ceiling: bigger is  
better  
owner = "PERF"  
test = "aura-perf::budget_hash_throughput"
```

```
[[budget]]  
id = "catalog_open_200k"  
what = "Open a catalog holding 200,000 photo_file rows"  
unit = "milliseconds"  
target = 250  
ceiling = 600  
owner = "SRC"  
test = "aura-perf::budget_catalog_open"
```

```
[[budget]]  
id = "peak_rss_import"
```

```

what = "Peak resident set size during the 4,000-file import"
unit = "MB"
target = 700
ceiling = 1200
owner = "PERF"
test = "aura-perf::budget_peak_rss"

[[budget]]
id = "resume_after_kill"
what = "Time from launch to first new task claimed after kill
-9"
unit = "seconds"
target = 2
ceiling = 5
owner = "SRC"
test = "aura-perf::budget_resume"

[[budget]]
id = "cold_launch"
what = "Process start to interactive window, warm filesystem
cache"
unit = "seconds"
target = 1.2
ceiling = 2.5
owner = "SFE"
test = "aura-perf::budget_cold_launch"

[[budget]]
id = "cancel_ack"
what = "Cancel pressed to run state = cancelled"
unit = "milliseconds"
target = 200
ceiling = 500
owner = "SFE"
test = "aura-perf::budget_cancel"

```

```
[[budget]]
id = "migration_200k"
what = "Apply a migration to a 200,000-row catalog, including
verified backup"
unit = "seconds"
target = 1
ceiling = 5
owner = "SRC"
test = "aura-perf::budget_migration"
```

```
[[budget]]
id = "grid_frame_time"
what = "Worst frame during a 10-second scroll of 4,000 cells"
unit = "milliseconds"
target = 8
ceiling = 16.7
owner = "SFE"
test = "ui::grid.perf.test.ts"
```

```
[[budget]]
id = "ipc_grid_payload"
what = "list_grid for 4,000 items, serialise plus transfer"
unit = "milliseconds"
target = 60
ceiling = 150
owner = "MBE"
test = "aura-perf::budget_grid_payload"
```

2. `crates/aura-perf/src/budget.rs` — budgets as assertions

```
use std::path::Path;

use serde::Deserialize;
```

```

/// A budget loaded from perf/budgets.toml. Deliberately not
`Copy`: a budget is
/// a document, and passing it by reference keeps the failure
message able to
/// quote `what` and `owner`.
#[derive(Debug, Clone, Deserialize)]
pub struct Budget {
    pub id: String,
    pub what: String,
    pub unit: String,
    pub target: f64,
    /// Upper bound. Present for latency-style budgets.
    pub ceiling: Option<f64>,
    /// Lower bound. Present for throughput-style budgets.
    pub floor: Option<f64>,
    pub owner: String,
    pub test: String,
}

#[derive(Debug, Clone, Deserialize)]
pub struct BudgetFile {
    pub meta: Meta,
    #[serde(rename = "budget")]
    pub budgets: Vec<Budget>,
}

#[derive(Debug, Clone, Deserialize)]
pub struct Meta {
    pub tier: String,
    pub ci_slack: f64,
    pub regression_warn_pct: f64,
    pub regression_fail_pct: f64,
}

impl BudgetFile {

```

```

pub fn load() -> anyhow::Result<Self> {
    let raw = std::fs::read_to_string(Path::new(env!("CAR
GO_MANIFEST_DIR")))
        .join("../perf/budgets.toml"))?;
    Ok(toml::from_str(&raw)?)
}

pub fn get(&self, id: &str) -> anyhow::Result<&Budget> {
    self.budgets.iter().find(|b| b.id == id)
        .ok_or_else(|| anyhow::anyhow!("no budget with id
{id}; add it to perf/budgets.toml"))
}
}

```

```

/// The assertion. Call it at the end of a measurement and it
/// either passes
/// quietly or fails with everything a reviewer needs to act.
///
/// Three outcomes, and the middle one matters most:
/// * inside target -> pass silently
/// * between target and ceiling -> pass, but print a WARN
line that CI turns
/// into a PR comment. Slow erosion is how budgets die, a
nd a warning that
/// is visible on the PR that caused it is the only warni
ng anyone reads.
/// * beyond ceiling -> panic, which fails the test
pub fn assert_budget(id: &str, measured: f64) {
    let file = BudgetFile::load().unwrap_or_else(|e| panic!
("cannot load budgets.toml: {e}"));
    let b = file.get(id).unwrap_or_else(|e| panic!("{e}"));
    let slack = if is_ci() { file.meta.ci_slack } else { 1.0
};
}

```

```

// Emit a machine-readable line on every run. CI collects
these into the

```

```

    // trend chart, so a graph exists without anyone maintain
    ing a spreadsheet.
    println!("AURA_BUDGET {{\"id\": \"{}\", \"measured\": {}, \"u
nit\": \"{}\", \"ci\": {}}}",
        b.id, measured, b.unit, is_ci());

    if let Some(ceiling) = b.ceiling {
        let limit = ceiling * slack;
        assert!(measured <= limit,
            "BUDGET EXCEEDED: {} \n id          {} \n measured
{measured} {} \n target  {} {} \n \
        ceiling {ceiling} {} (x{slack} on CI = {limit})
 \n owner    {} \n \
        This is a build failure, not a warning. Either m
ake it faster or \
        change the budget in perf/budgets.toml with an A
DR explaining why.",
            b.what, b.id, b.unit, b.target, b.unit, b.unit,
b.owner);
        if measured > b.target * slack {
            println!("::warning::BUDGET WARN {} measured {mea
sured}} over target {} (owner {})",
                b.id, b.unit, b.target, b.unit, b.owner);
        }
    }

    if let Some(floor) = b.floor {
        let limit = floor / slack;
        assert!(measured >= limit,
            "BUDGET BELOW FLOOR: {} \n id          {} \n measure
d {measured} {} \n floor    {floor} {} \
        ({slack} on CI = {limit}) \n owner    {}",
            b.what, b.id, b.unit, b.unit, b.owner);
        if measured < b.target / slack {
            println!("::warning::BUDGET WARN {} measured {mea
sured}} under target {} (owner {})",

```

```

        b.id, b.unit, b.target, b.unit, b.owner);
    }
}

fn is_ci() -> bool { std::env::var("CI").is_ok_and(|v| v ==
"true" || v == "1") }

/// Peak RSS, measured properly on each platform. `/proc/self/status` VmHWM on
/// Linux, `task_info` on macOS, `GetProcessMemoryInfo` on Windows. Sampling
/// `ps` in a loop would miss a spike between samples, and a spike is exactly
/// what an OOM is.
pub fn peak_rss_mb() -> anyhow::Result<f64> {
    #[cfg(target_os = "linux")]
    {
        let status = std::fs::read_to_string("/proc/self/status")?;
        for line in status.lines() {
            if let Some(rest) = line.strip_prefix("VmHWM:") {
                let kb: f64 = rest.trim().trim_end_matches("kB").trim().parse()?;
                return Ok(kb / 1024.0);
            }
        }
        anyhow::bail!("VmHWM not found in /proc/self/status")
    }
    #[cfg(not(target_os = "linux"))]
    { platform::peak_rss_mb() }
}

```

The eight budget tests


```

// crates/aura-perf/tests/budgets.rs
use aura_perf::budget::{assert_budget, peak_rss_mb};

#[test]
#[ignore = "perf: needs the Full corpus, runs in the nightly lane"]
fn budget_import_4000() {
    let fixture = ensure_corpus(Corpus::Full);
    let cat = fresh_catalog();
    let t0 = std::time::Instant::now();
    let report = import_folder(&cat, fixture.path()).expect(
("import"));
    let secs = t0.elapsed().as_secs_f64();

    // Assert the work actually happened before asserting how
    fast it was. A
    // timing test that passes because the function returned
    early is worse
    // than no test at all.
    assert_eq!(report.files_imported, 4000);
    assert_eq!(report.files_quarantined, 0);
    assert_budget("import_4000_files", secs);
}

#[test]
#[ignore = "perf: nightly"]
fn budget_hash_throughput() {
    let files = ensure_corpus(Corpus::Full).files();
    let bytes: u64 = files.iter().map(|f| f.size_bytes).sum
    ();
    let t0 = std::time::Instant::now();
    let results = aura_ingest::hash::hash_batch(&files, 8);
    let secs = t0.elapsed().as_secs_f64();
    assert!(results.iter().all(|(_, r)| r.is_ok()), "all file
s must hash");
}

```

```

    assert_budget("hash_throughput", (bytes as f64) / secs /
1_048_576.0);
}

#[test]
#[ignore = "perf: nightly"]
fn budget_catalog_open() {
    let path = synthesise_catalog_with_rows(200_000); // bu
ilt once, cached
    // Five opens, report the median. A single cold open meas
ures the page cache,
    // not our code.
    let mut samples = Vec::new();
    for _ in 0..5 {
        let t0 = std::time::Instant::now();
        let cat = aura_catalog::open(&path, test_clock(), "pe
rf").expect("open");
        samples.push(t0.elapsed().as_secs_f64() * 1000.0);
        drop(cat);
    }
    samples.sort_by(f64::total_cmp);
    assert_budget("catalog_open_200k", samples[2]);
}

#[test]
#[ignore = "perf: nightly"]
fn budget_peak_rss() {
    let fixture = ensure_corpus(Corpus::Full);
    let cat = fresh_catalog();
    import_folder(&cat, fixture.path()).expect("import");
    assert_budget("peak_rss_import", peak_rss_mb().expect("rs
s"));
}

#[test]
#[ignore = "perf: nightly, spawns a child process"]

```

```

fn budget_resume() {
    let dir = fresh_fixture_copy(Corpus::Small);
    let child = spawn_import_child(&dir, KillPoint::MidHashBatch);
    child.wait_until_signal(KillPoint::MidHashBatch);
    child.kill_9();

    let t0 = std::time::Instant::now();
    let cat = aura_catalog::open(&dir.join("catalog.sqlite"),
test_clock(), "perf").expect("open");
    aura_jobs::reclaim::recover_on_startup(&cat).expect("recover");
    let first = claim_first_task(&cat).expect("claim");
    let secs = t0.elapsed().as_secs_f64();
    assert!(first.is_some(), "resume must find work to do");
    assert_budget("resume_after_kill", secs);
}

#[test] #[ignore = "perf: nightly"] fn budget_cold_launch() {
/* spawn the built binary, wait for the ready IPC event */ }
#[test] #[ignore = "perf: nightly"] fn budget_cancel() { /*
4,000-file import, cancel, measure to state = cancelled */ }
#[test] #[ignore = "perf: nightly"] fn budget_migration() { /*
* 200k rows, v0 -> v1, includes the verified backup */ }
#[test] #[ignore = "perf: nightly"] fn budget_grid_payload()
{ /* list_grid(4000), serialise + transfer */ }

```

3. Local gates — **make check** before anything else

```

#!/usr/bin/env bash
# scripts/check.sh – the exact set of gates CI runs on a PR,
runnable locally.
#
# The contract: if this passes locally, the PR lane cannot fail
on style,

```

```

# lint, banned patterns, contract drift or the fast test suite. Anything that
# CI checks and this does not is a bug in this script.
set -euo pipefail

START=$(date +%s)
FAILED=0

step() {
    local name="$1"; shift
    printf '\n\033[1m==> %s\033[0m\n' "$name"
    if "$@"; then
        printf '\033[32m    ok\033[0m\n'
    else
        printf '\033[31m    FAILED: %s\033[0m\n' "$name"
        FAILED=1
    fi
}

# Ordered cheapest-first so a formatting mistake costs three
# seconds, not
# eleven minutes. Every step runs even if an earlier one fails,
# so one run
# reports every problem instead of one problem at a time.
step "format" cargo fmt --all -- --check
step "banned patterns" ./scripts/check-banned.sh
step "contract drift" cargo xtask contracts --check
step "ipc type parity" cargo xtask ipc-types --check
step "clippy" cargo clippy --workspace --all-targets --all-features -- -D warnings
step "build" cargo build --workspace --all-targets
step "unit + property" cargo test --workspace --lib
step "integration" cargo test --workspace --test '*'
step "doc tests" cargo test --workspace --doc
step "deny" cargo deny check

```

```

step "typescript"      bash -c 'cd ui && npm run -s typeche
ck'
step "eslint"          bash -c 'cd ui && npm run -s lint'
step "vitest"          bash -c 'cd ui && npm run -s test --
--run'
step "runbook coverage" cargo xtask runbooks --check

ELAPSED=$(( $(date +%s) - START ))
printf '\n\033[1mtotal %ds\033[0m\n' "$ELAPSED"

if [ "$FAILED" -ne 0 ]; then
    printf '\033[31mcheck failed \u2014 fix the steps marked FA
ILED above\033[0m\n'
    exit 1
fi
printf '\033[32mall gates passed\033[0m\n'

```

Gate	Typical time	Blocks merge	What it catches that a human review would not
<code>cargo fmt --check</code>	2 s	Yes	Diff noise that hides real changes in review
<code>check-banned.sh</code>	3 s	Yes	A new <code>unwrap</code> or <code>HashMap</code> slipped in during a refactor
<code>xtask contracts -check</code>	1 s	Yes	A frozen contract edited without an ADR
<code>xtask ipc-types -check</code>	2 s	Yes	Rust and TypeScript drifting apart silently
<code>clippy -D warnings</code>	90 s	Yes	Indexing that can panic, float comparison, integer division
Unit + property	80 s	Yes	Invariant violations across thousands of generated inputs
Integration	4 min	Yes	Crates that work alone and break together

Gate	Typical time	Blocks merge	What it catches that a human review would not
<code>cargo deny check</code>	5 s	Yes	A GPL transitive dependency, a banned crate, a known advisory
<code>xtask runbooks --check</code>	1 s	Yes	A new error code with no runbook, i.e. a future support ticket
Chaos	25 min	Nightly only	Data loss under kill, yank and full disk
Budgets	20 min	Nightly only	Performance erosion that no reviewer can see in a diff

4. `.github/workflows/ci.yml` — the PR lane

```

name: CI

on:
  pull_request:
  push:
    branches: [main]

# A new push cancels the previous run for the same ref. Without this, a branch
# pushed four times in ten minutes burns four full matrices and the last result
# arrives after the developer has moved on.
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

env:
  CARGO_TERM_COLOR: always
  CARGO_INCREMENTAL: 0          # incremental builds are slower in ephemeral CI
  RUST_BACKTRACE: 1

```

```

CI: "true"

jobs:
  # -----
  ----- stage 1
    # Cheap gates on one runner. Nothing expensive starts until
    # these pass, so a
    # missing semicolon costs 90 seconds of compute rather than
    # 40 minutes.
    quick:
      name: quick gates
      runs-on: ubuntu-latest
      timeout-minutes: 15
      steps:
        - uses: actions/checkout@v4
        - uses: dtolnay/rust-toolchain@master
          with:
            toolchain: "1.83.0"          # pinned; must equal r
            components: rustfmt, clippy
        - uses: Swatinem/rust-cache@v2
          with: { shared-key: "quick" }

        - name: toolchain pin matches rust-toolchain.toml
          run: |
            PINNED=$(grep -oP 'channel\s*=\s*"K[^\"]+' rust-too
            lchain.toml)
            ACTUAL=$(rustc --version | awk '{print $2}')
            test "$PINNED" = "$ACTUAL" || { echo "toolchain dri
            ft: $PINNED vs $ACTUAL"; exit 1; }

        - run: cargo fmt --all -- --check
        - run: ./scripts/check-banned.sh
        - run: cargo xtask contracts --check
        - run: cargo xtask ipc-types --check
        - run: cargo xtask runbooks --check

```

```
- run: cargo clippy --workspace --all-targets --all-features -- -D warnings
```

licences:

```
name: licences and advisories
```

```
runs-on: ubuntu-latest
```

```
timeout-minutes: 10
```

steps:

```
- uses: actions/checkout@v4
```

```
- uses: EmbarkStudios/cargo-deny-action@v2
```

```
with: { command: check }
```

frontend:

```
name: frontend
```

```
runs-on: ubuntu-latest
```

```
timeout-minutes: 15
```

steps:

```
- uses: actions/checkout@v4
```

```
- uses: actions/setup-node@v4
```

```
with: { node-version: "20", cache: "npm", cache-dependency-path: ui/package-lock.json }
```

```
dependency-path: ui/package-lock.json }
```

```
- run: npm ci
```

```
working-directory: ui
```

```
- run: npm run typecheck
```

```
working-directory: ui
```

```
- run: npm run lint
```

```
working-directory: ui
```

```
- run: npm run test -- --run --coverage
```

```
working-directory: ui
```

```
# -----
```

```
----- stage 2
```

test:

```
name: test ${ matrix.os }
```

```
needs: [quick]
```

```
strategy:
```



```

    fail-fast: false                # a Windows-only failure must not hide a macOS one
    matrix:
      os: [ubuntu-latest, windows-latest, macos-14]
    runs-on: ${matrix.os}
    timeout-minutes: 45
    steps:
      - uses: actions/checkout@v4
      - uses: dtolnay/rust-toolchain@master
        with: { toolchain: "1.83.0" }
      - uses: Swatinem/rust-cache@v2
        with: { shared-key: "test-${matrix.os}" }

      # Fixtures are generated, never downloaded. Same seed on every OS, so a
      # platform-specific failure is a real platform difference.

      - name: generate fixtures
        run: cargo xtask fixtures --corpus small --seed 42
      - name: generate hostile fixtures
        run: cargo xtask fixtures --corpus hostile --seed 42

      - name: fixture digest is identical across platforms
        run: cargo xtask fixtures --verify-digest

      - run: cargo build --workspace --all-targets
      - run: cargo test --workspace --lib
      - run: cargo test --workspace --test '*'
      - run: cargo test --workspace --doc

      - name: no test was silently skipped
        shell: bash
        run: |
          # A test that stops being compiled is a test that stops protecting us.
          COUNT=$(cargo test --workspace -- --list 2>/dev/null

```

```

l | grep -c ': test$')
    echo "discovered $COUNT tests"
    test "$COUNT" -ge 220 || { echo "expected at least
220 tests, found $COUNT"; exit 1; }

- uses: actions/upload-artifact@v4
  if: failure()
  with:
    name: failure-logs-${{ matrix.os }}
    path: |
      target/debug/deps/*.log
      tests/artifacts/**
    retention-days: 14

coverage:
  name: coverage
  needs: [quick]
  runs-on: ubuntu-latest
  timeout-minutes: 40
  steps:
    - uses: actions/checkout@v4
    - uses: dtolnay/rust-toolchain@master
      with: { toolchain: "1.83.0", components: llvm-tools-p
review }
    - uses: taiki-e/install-action@cargo-llvm-cov
    - run: cargo xtask fixtures --corpus small --seed 42
    - name: coverage with a floor
      run: |
        cargo llvm-cov --workspace --lcov --output-path lco
v.info \
        --ignore-filename-regex '(xtask|tests)/' \
        --fail-under-lines 85
    - uses: actions/upload-artifact@v4
      with: { name: coverage, path: lcov.info }

build-app:

```

```

name: package ${ matrix.os }
needs: [quick, frontend]
strategy:
  matrix:
    os: [ubuntu-latest, windows-latest, macos-14]
runs-on: ${ matrix.os }
timeout-minutes: 45
steps:
  - uses: actions/checkout@v4
  - uses: dtolnay/rust-toolchain@master
    with: { toolchain: "1.83.0" }
  - uses: actions/setup-node@v4
    with: { node-version: "20", cache: "npm", cache-dependency-path: ui/package-lock.json }
  - name: linux webkit deps
    if: runner.os == 'Linux'
    run: |
      sudo apt-get update
      sudo apt-get install -y libwebkit2gtk-4.1-dev libappindicator3-dev librsvg2-dev patchelf
  - run: npm ci
    working-directory: ui
  - run: npm run build
    working-directory: ui
  - run: cargo build --release -p aura-app

  # The app must start and answer one IPC call. A binary
  # that builds but
  # does not launch has passed every other gate in this file.
  - name: smoke test the built binary
    run: cargo run --release -p aura-cli -- smoke --timeout-secs 30

required:
  name: all required checks

```

```

if: always()
needs: [quick, licences, frontend, test, coverage, build-
app]
runs-on: ubuntu-latest
steps:
  # One job for branch protection to require. Without thi
s, adding a job to
  # the matrix silently makes it optional, which is how a
green button starts
  # meaning nothing.
  - name: fail if any dependency failed
    run: |
      echo '${{ toJSON(needs) }}' | grep -q '"result": "f
ailure"' && exit 1
      echo '${{ toJSON(needs) }}' | grep -q '"result": "c
ancelled"' && exit 1
      echo "all required checks passed"

```

5. `.github/workflows/nightly.yml` — chaos and budgets

```

name: Nightly

on:
  schedule:
    - cron: "0 18 * * *"      # 18:00 UTC = 23:45 Asia/Kathmand
u, after the workday
  workflow_dispatch:
    inputs:
      corpus:
        description: "Corpus size for perf runs"
        default: "full"

jobs:

```

```

chaos:
  name: chaos ${ matrix.os }
  strategy:
    fail-fast: false
  matrix:
    os: [ubuntu-latest, windows-latest, macos-14]
  runs-on: ${ matrix.os }
  timeout-minutes: 90
  steps:
    - uses: actions/checkout@v4
    - uses: dtolnay/rust-toolchain@master
      with: { toolchain: "1.83.0" }
    - run: cargo xtask fixtures --corpus small --seed 42
    - name: chaos suite
      run: cargo test --workspace --test 'chaos*' -- --igno
red --test-threads 1
    - uses: actions/upload-artifact@v4
      if: always()
      with:
        name: chaos-artifacts-${ matrix.os }
        path: tests/artifacts/**

budgets:
  name: performance budgets
  # A self-hosted runner pinned to known hardware. Budget n
umbers from a
  # shared cloud runner vary by 40 % between runs, which ma
kes a 7 %
  # regression gate meaningless.
  runs-on: [self-hosted, aura-perf, t-typ]
  timeout-minutes: 120
  steps:
    - uses: actions/checkout@v4
    - uses: dtolnay/rust-toolchain@master
      with: { toolchain: "1.83.0" }
    - run: cargo xtask fixtures --corpus ${ inputs.corpus

```

```

|| 'full' }} --seed 42
  - name: run budget assertions
    run: cargo test --release -p aura-perf -- --ignored -
-test-threads 1 --nocapture | tee perf.log
  - name: compare against the stored baseline
    run: cargo xtask perf-compare --input perf.log --base
line perf/baseline.json
  - uses: actions/upload-artifact@v4
    with: { name: perf-log, path: perf.log }
  - name: open an issue on regression
    if: failure()
    uses: actions/github-script@v7
    with:
      script: |
        github.rest.issues.create({
          owner: context.repo.owner, repo: context.repo.r
epo,
          title: `Perf regression on ${new Date().toISOString().slice(0,10)}`,
          labels: ['perf', 'regression'],
          body: 'Nightly budget run failed. See the perf-
log artifact.'
        })

flake-hunt:
  name: flake hunt
  runs-on: ubuntu-latest
  timeout-minutes: 90
  steps:
    - uses: actions/checkout@v4
    - uses: dtolnay/rust-toolchain@master
      with: { toolchain: "1.83.0" }
    - run: cargo xtask fixtures --corpus small --seed 42
    - name: run the suite ten times
      run: |
        # A test that fails one run in ten is a test nobody

```

```

trusts, and an
    # untrusted suite gets ignored on the day it finds
a real bug.
    for i in $(seq 1 10); do
        echo "--- run $i"
        cargo test --workspace || { echo "FLAKE on run
$i"; exit 1; }
    done

```

6. **cargo xtask perf-compare** — regression detection

```

// xtask/src/perf_compare.rs

/// Compare measured values against a committed baseline and
enforce the two
/// regression thresholds from budgets.toml.
///
/// The baseline is a committed file, not a database. That me
ans a regression
/// shows up as a diff in a pull request, reviewed by a perso
n, rather than as a
/// silent update to a dashboard nobody opens.
pub fn run(input: &Path, baseline_path: &Path) -> anyhow::Res
ult<()> {
    let budgets = BudgetFile::load()?;
    let measured = parse_budget_lines(&std::fs::read_to_strin
g(input)?)?;
    let baseline: BTreeMap<String, f64> =
        serde_json::from_str(&std::fs::read_to_string(baselin
e_path)?)?;

    let mut failures = Vec::new();
    let mut warnings = Vec::new();

    for (id, now) in &measured {

```

```

        let Some(before) = baseline.get(id) else {
            warnings.push(format!("{id}: no baseline yet, recording {now}"));
            continue;
        };
        let budget = budgets.get(id)?;
        // Direction matters: for a floor budget, smaller is
a regression.
        let change_pct = if budget.floor.is_some() {
            (before - now) / before * 100.0
        } else {
            (now - before) / before * 100.0
        };

        if change_pct > budgets.meta.regression_fail_pct {
            failures.push(format!(
                "{id}: {before} -> {now} {} ({{change_pct:+.1}}
%) owner {}",
                budget.unit, budget.owner));
        } else if change_pct > budgets.meta.regression_warn_p
ct {
            warnings.push(format!(
                "{id}: {before} -> {now} {} ({{change_pct:+.1}}
%) owner {}",
                budget.unit, budget.owner));
        } else if change_pct < -budgets.meta.regression_warn_
pct {
            // An improvement large enough to be suspicious.
Usually it means the
            // test stopped doing the work. Say so out loud.
            warnings.push(format!(
                "{id}: improved by {:.1} % \u2014 verify the
test still does the work",
                -change_pct));
        }
    }
}

```



```

    for w in &warnings { println!("::warning::PERF {w}"); }
    if !failures.is_empty() {
        for f in &failures { println!("::error::PERF REGRESSI
ON {f}"); }
        anyhow::bail!("{}", performance regression(s) beyond {}
%",
            failures.len(), budgets.meta.regression_fail_pc
t);
    }
    println!("no regressions beyond {} %", budgets.meta.regre
ssion_warn_pct);
    Ok(())
}

```

7. Merge protection and the flake policy

Rule	Setting	Reason
Required check	Only <code>all required checks</code>	Adding a matrix entry cannot silently make it optional
Linear history	Squash merge only	<code>git bisect</code> over a phase boundary stays usable
Up-to-date branch	Required	Two green PRs that break when combined is the classic Friday outage
Force push to <code>main</code>	Blocked for everyone	The contract digests in <code>contracts.lock</code> depend on history
Flaky test	Quarantine within 24 h with an owner and an issue; deleted if unfixed in 7 days	A permanently quarantined test is a lie that costs CI minutes
Budget change	Requires an ADR and a second reviewer	Otherwise the easiest fix for a slow build is a bigger budget
Nightly red	Blocks the next phase from starting	Phase 02 building on a broken phase 01 costs a week

8. Acceptance criteria for T18

- ☐ `perf/budgets.toml` contains all 10 budgets; every `test` field names a test that exists
- ☐ `assert_budget` fails the build past a ceiling and past a floor; both directions demonstrated by temporarily setting an impossible value, output pasted
- ☐ `assert_budget` prints a `::warning::` line between target and ceiling, verified
- ☐ Every perf test asserts that the work happened before asserting how fast it was
- ☐ `peak_rss_mb` returns a sane value on all three platforms; the three numbers are pasted
- ☐ `./scripts/check.sh` runs every PR-lane gate locally and reports all failures in one pass, not just the first
- ☐ PR lane green on Linux, Windows and macOS; total wall time under 20 minutes with a warm cache; time pasted
- ☐ Stage 2 does not start until stage 1 passes, proven by a deliberately misformatted commit
- ☐ The toolchain-drift check fails when `rust-toolchain.toml` and the workflow disagree, demonstrated
- ☐ Fixture digest verification passes identically on all three platforms
- ☐ The test-count floor fails when a test file is commented out, demonstrated
- ☐ `all required checks` fails when any dependency fails, demonstrated with a deliberate failure
- ☐ Nightly chaos lane green on all three platforms
- ☐ Nightly budget lane green on the pinned self-hosted runner; `perf/baseline.json` committed with the first run's numbers
- ☐ `perf-compare` fails on a 15 % regression and warns on 8 %, both demonstrated with a synthetic log
- ☐ Flake hunt: 10 consecutive full runs, zero failures; log pasted

☐ Branch protection configured exactly as the table above; a screenshot or the API output is attached to the phase-exit report

9. Claude Code prompt for T18

Act as `DEVOPS` with `PERF` reviewing. Implement T18 of PHASE-01 from this page.

Create `perf/budgets.toml` verbatim, `crates/aura-perf/src/budget.rs` with the three-outcome `assert_budget` and real per-platform `peak_rss_mb`, all ten budget tests (write the four sketched ones in full), `scripts/check.sh`, `.github/workflows/ci.yml`, `.github/workflows/nightly.yml`, and `cargo xtask perf-compare`.

Hard constraints: budgets live in exactly one file and no number is duplicated in Rust; the ceiling assertion fails the build, it does not warn; every perf test asserts the work happened before it asserts the timing; CI stage 2 must not start before stage 1 passes; the toolchain version is pinned in exactly one place and CI verifies it matches; fixtures are generated in CI, never downloaded; branch protection requires exactly one aggregate check.

Then prove each gate can fail: mis-format a file, add an `unwrap`, edit a frozen contract, change `ui/src/ipc/types.ts` by hand, comment out a test file, and set one budget to an impossible value. Paste all six failing outputs, then revert them.

Evidence I expect pasted back: a green PR-lane run link with total wall time; the six deliberate-failure outputs; `peak_rss_mb` on all three platforms; the first `perf/baseline.json`; the `perf-compare` regression and warning demonstrations; and the 10-run flake log. Stop after T18.